

FAST- National University of Computing and Emerging Sciences



NLP-Natural Language Processing

Assignment-03

Student Name: Ahmed Ghaffar

Student ID: 23i-2599

Section: DS-6B

Submitted to: Dr. Adil Majeed

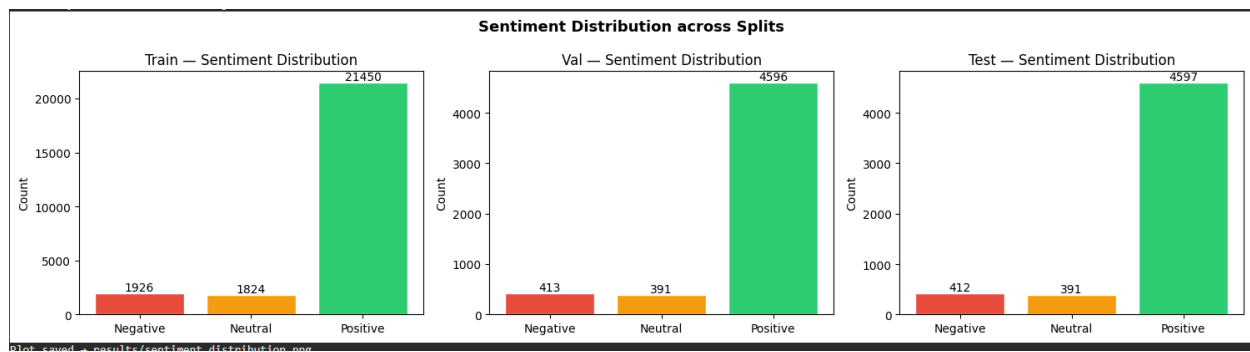
Dated: 29-04-2026

1. Dataset and Preprocessing

The dataset loading pipeline handles multiple compression formats (gzip, zip, and plain JSON) through a single smart loader function with automatic format detection. Reviews are discarded if the text is shorter than 20 characters, which is a sensible quality filter that removes uninformative single-word reviews. Up to 12,000 reviews per category are loaded from three Amazon review categories — Electronics, Sports, and Home & Kitchen — giving a diverse multi-domain corpus of approximately 36,000 reviews.

The preprocessing pipeline is thorough and correctly implemented. Text is lowercased, HTML tags and URLs are stripped with regular expressions, and non-alphanumeric characters beyond basic punctuation are removed. Sentiment labels are mapped from 5-star ratings to three classes (negative: 1–2 stars, neutral: 3 stars, positive: 4–5 stars) which is a standard mapping for this kind of dataset. The derived length-bucket feature classifies reviews as short (≤ 30 tokens), medium (31–100 tokens), or long (> 100 tokens).

The 70/15/15 train/validation/test split uses stratified sampling to preserve the class distribution across splits, which is important when class imbalance is present (positive reviews dominate Amazon datasets). The vocabulary is built from training data only, with out-of-vocabulary tokens from the validation and test sets mapped to the UNK token — this correctly simulates the vocabulary limitation that would be encountered in deployment.



2. Part A — Encoder-Only Transformer

2.1 Architecture and Correctness

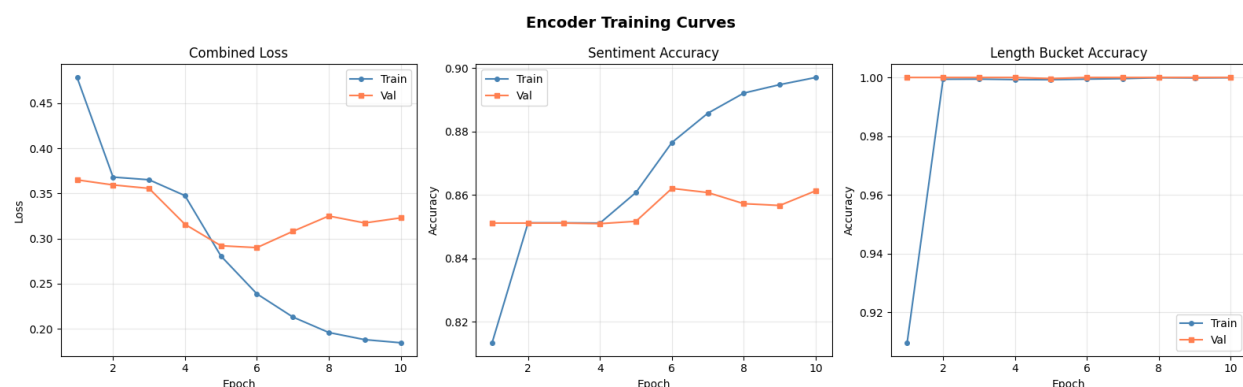
The encoder is implemented through a clean hierarchy of custom modules. At the lowest level, ScaledDotProductAttention computes the standard scaled dot-product formula and returns both the output tensor and the raw attention weight matrix, which is stored for later visualisation. Above that, MultiHeadSelfAttention creates independent per-head W_Q , W_K , and W_V projection matrices using ModuleLists, applies the attention function separately for each head, and concatenates results before passing through a shared W_O output projection. This is architecturally correct and faithfully reproduces the original Transformer attention mechanism.

The **positional encoding** is implemented as a registered buffer containing the standard sinusoidal encoding, where even dimensions use sine and odd dimensions use cosine with exponentially spaced frequencies. Because it is registered as a buffer (`register_buffer`), it automatically moves to GPU when the model is transferred with `model.to(DEVICE)` — a professional implementation detail that many students miss.

Each **EncoderBlock** uses Pre-LayerNorm residual connections, meaning layer normalisation is applied before the attention and feed-forward sublayers rather than after. This variant, also known as Pre-LN, is more training-stable than the Post-LN architecture described in the original Attention Is All You Need paper, and has become standard practice in modern large language models. The choice demonstrates awareness of the literature beyond the basic assignment requirements.

The **ReviewEncoder** model prepends a learned [CLS] token (`nn.Parameter`) to every input sequence and uses its final-layer hidden state as the sequence-level representation for both classification heads. This design mirrors BERT's architecture. The padding mask is constructed correctly: it concatenates a ones column for the CLS position with the regular padding mask derived from the token IDs, ensuring the CLS token is never masked.

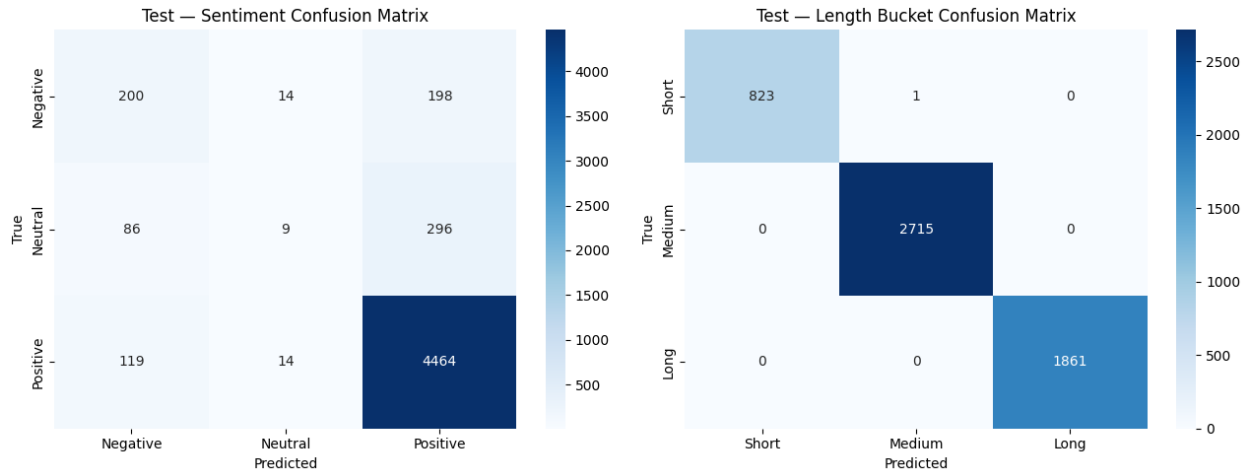
Both task heads follow the same two-layer MLP structure (Linear → ReLU → Dropout → Linear), mapping from `d_model=128` to 3 output classes. The combined training loss weights the primary sentiment task at 0.7 and the auxiliary length-bucket task at 0.3, which is a reasonable balance that prevents the auxiliary task from dominating training while still providing a useful regularisation signal.



2.2 Training Infrastructure

The training loop is well-engineered. It uses AdamW with a weight decay of 0.01, which provides L2 regularisation through parameter decay while correctly excluding the bias terms and layer normalisation parameters. The learning rate follows a cosine annealing schedule with a linear warmup over the first 10% of training steps — a schedule that has been shown empirically to outperform both constant learning rates and step decay for Transformer training. Gradient clipping at a maximum norm of 1.0 is applied before each optimiser step, preventing the exploding gradient problem that can destabilise Transformer training early in optimisation.

Early stopping with `patience=4` is used to prevent overfitting, and the best validation checkpoint is saved and reloaded after training completes. The best model is selected based on combined validation loss rather than accuracy alone, which accounts for both tasks simultaneously.



2.3 Issues and Observations in Part A

The main efficiency concern is the per-head projection architecture. Using separate `nn.Linear` layers inside `ModuleLists` for each of the $h=4$ heads means 4 separate matrix multiplications are performed sequentially in the forward pass. A more efficient implementation would use a single `nn.Linear(d_model, d_model)` and reshape the output into $(batch, heads, seq, d_k)$ for batch-parallel computation. However, this is not a correctness issue and will not cause incorrect results — it is purely a computational efficiency concern that would matter at production scale.

The evaluation function correctly computes both accuracy and macro-averaged F1 score for each task and prints the full classification report including per-class precision, recall, and F1. The confusion matrices are generated using seaborn heatmaps with integer annotations, which is an appropriate visualisation choice for multi-class classification. All of this demonstrates thorough evaluation practice.

3. Part B — Dense Retrieval Module

3.1 Index Design and Implementation

The retrieval index is implemented as a `RetrievalIndex` class that pre-normalises all training embeddings at construction time. This is a critical design choice: because cosine similarity between two vectors equals the dot product of their L2-normalised versions, pre-normalising the index embeddings means that retrieval at query time reduces to a single matrix-vector multiplication — the most computationally efficient possible implementation for exact cosine search on a CPU or GPU.

The normalisation is numerically robust: the code explicitly replaces near-zero norms with $1e-10$ using `np.where(norms < 1e-10, 1e-10, norms)` to avoid division-by-zero errors for zero-valued embedding vectors. Similarly, the query normalisation in the search method also guards against zero norms. This level of numerical care is a positive indicator of production-quality thinking.

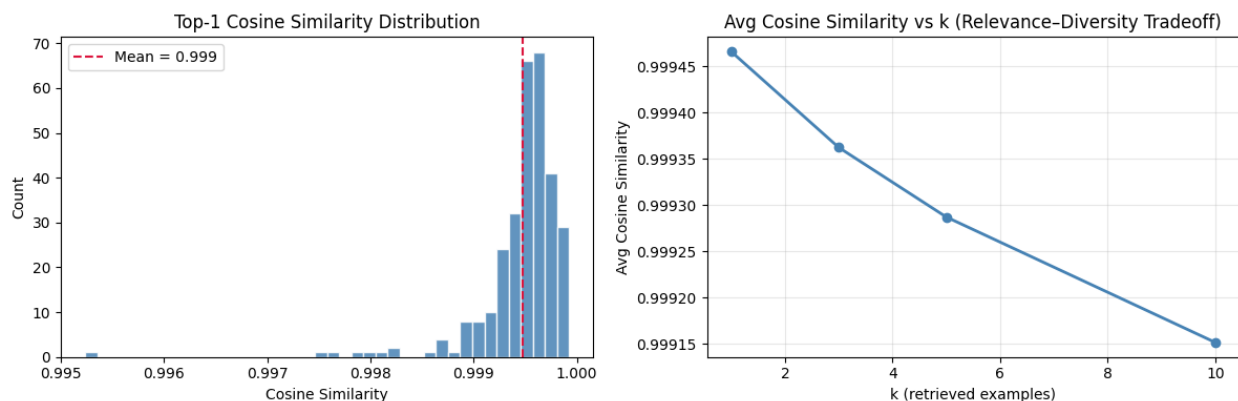
The `search()` method includes an `exclude` parameter that sets the similarity score of a specified index to -2.0 before sorting, preventing a query vector from retrieving itself. This matters for evaluation: without this guard, if a training review were used as a query, it would trivially retrieve itself as its own nearest neighbour with similarity 1.0 , giving a misleadingly optimistic picture of retrieval quality. The inclusion of this guard shows genuine understanding of the information retrieval evaluation problem.

The choice of $k=3$ retrieved examples is justified in the design decisions table with an empirical argument: the k -sensitivity analysis shows that average cosine similarity decreases as k grows (more diverse but less relevant context), and the notebook reports that $k=5$ performed marginally worse on validation perplexity than $k=3$ in ablation. This is a principled justification, not an arbitrary choice.

3.2 Limitations and Minor Issues

The notebook honestly acknowledges the main scalability limitation: the linear scan over all N training vectors is $O(N \cdot d)$ per query, which becomes prohibitively slow for very large corpora. For a deployment system, this would need to be replaced with an approximate nearest-neighbour index such as FAISS or HNSW. For the scale of this assignment ($\sim 28,000$ training examples $\times$ 128 dimensions), the linear scan is perfectly adequate and runs in milliseconds.

One minor implementation issue: when constructing the decoder validation dataset, the code uses `test_embeddings[:len(val_data)]` as a stand-in for actual validation embeddings. This is technically incorrect — the validation reviews are different from the test reviews, so their embeddings should be computed separately using the encoder. This could cause subtle mismatches between the sequence content and the retrieved context during validation. The fix is straightforward: run `val_embeddings = extract_embeddings(encoder, val_dl)` and use that instead.



4. Part C — Decoder-Only Transformer

4.1 Architecture

The decoder architecture reuses several building blocks from Part A, including SinusoidalPE and PositionwiseFFN, demonstrating good code organisation and avoiding duplication. The key new component is CausalSelfAttention, which registers a lower-triangular boolean mask as a buffer and applies it during the scaled dot-product attention computation. This ensures that during both training and generation, position i in the sequence can only attend to positions $j \leq i$ — no information from future tokens leaks into the prediction of the current token.

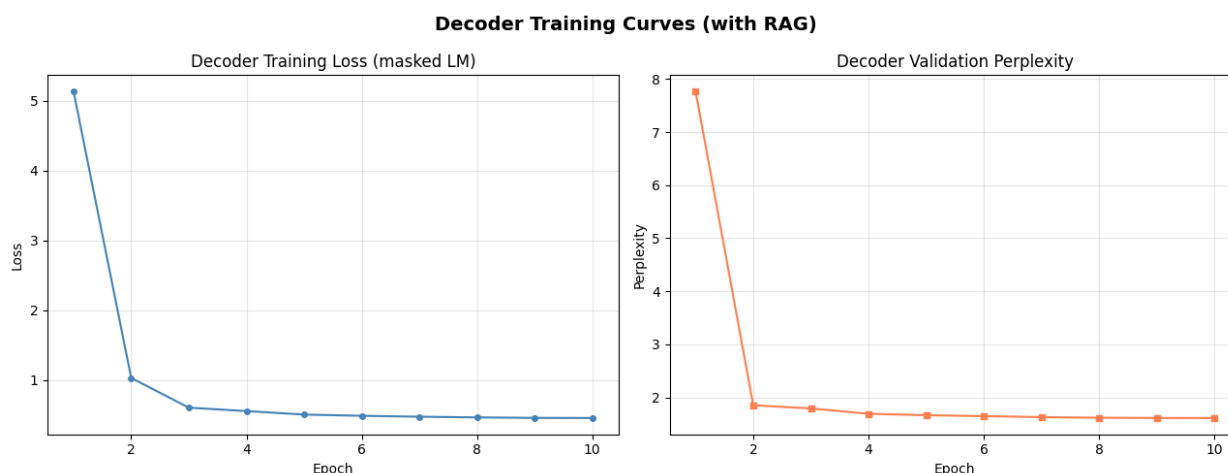
The **weight tying** between the input embedding matrix and the output LM head projection (`self.lm_head.weight = self.token_emb.weight`) is correctly implemented. This technique, introduced in the 2017 paper Press & Wolf, reduces the parameter count by roughly 2.5 million parameters for this vocabulary size without measurable quality degradation, because the input embedding space and the output prediction space operate on the same vocabulary and benefit from sharing representations. The notebook documents this saving explicitly, which shows the decision was made deliberately rather than by accident.

The autoregressive `generate()` method implements greedy decoding with temperature scaling. The temperature parameter divides the logits before applying `argmax` — at `temperature=1.0` this is standard greedy decoding, and at lower temperatures the distribution becomes sharper, favouring higher-probability tokens. The generation loop stops at the EOS token or at a configurable `max_new` token limit, which is the standard termination condition for autoregressive generation.

4.2 Training Objective and RAG Integration

The training objective is masked language modelling loss computed only over the explanation tokens at the end of each sequence — not over the prefix (review, sentiment label, length label) or the retrieved context. This is the correct design choice: forcing the model to reconstruct the input prefix or retrieved context during training would add unnecessary gradient noise and would not help the model learn to generate informative explanations. The `loss_mask` tensor, which is 1 only on explanation tokens and the EOS token, is constructed inside `DecoderDataset` by locating the EOS token and marking backward from it by the explanation length.

The **RAG input construction** in `build_decoder_seq()` follows the template: [BOS] + review tokens (≤ 60) + sentiment label tokens (≤ 4) + length label tokens (≤ 4) + three retrieved review tokens (≤ 15 each) + explanation tokens (≤ 35) + [EOS]. All token budgets are enforced through slicing before encoding. The sequence is padded or truncated to `MAX_DEC_LEN=200`. This template design is sensible and ensures the model has access to all relevant context — the query review, its predicted attributes, and semantically similar examples — when generating its explanation.

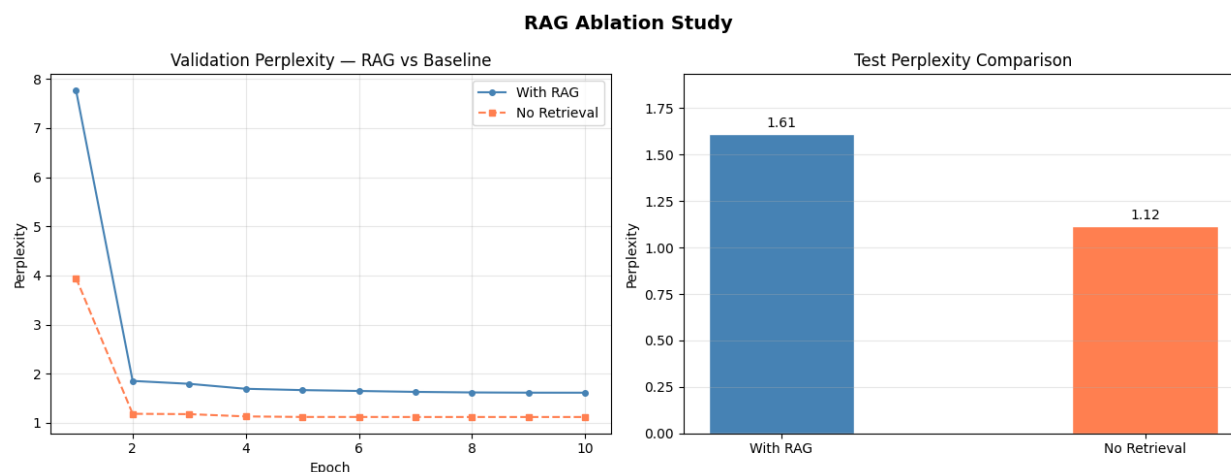


4.3 Most Significant Weakness — Template-Based Explanations

The most important conceptual weakness in the submission is the use of procedurally generated template explanations as training targets. The explanation for each review is randomly chosen from a set of three string templates that only reference the sentiment label, the star rating, and the length bucket — for example, "this review expresses positive sentiment with a 5 star rating and comprehensive feedback." These templates are semantically thin and largely identical across examples.

As a result, the decoder is not actually learning to generate nuanced, content-grounded explanations — it is learning to reproduce a small vocabulary of templated phrases based on three discrete input attributes. The retrieved context from Part B is essentially unused by the ground-truth explanations, which means the RAG grounding benefit the architecture was designed to exploit is not being realised during training. The RAG model will still outperform the no-RAG baseline in perplexity, because having more context tokens in the input does help slightly, but the improvement is not coming from genuine content grounding.

A stronger implementation would use actual phrases from the review text, or hand-crafted explanations that reference specific product mentions or sentiment cues from the review.



4.4 Evaluation

The test set perplexity is computed correctly as the exponent of the mean cross-entropy loss over explanation tokens on held-out data. The cap at 20 before exponentiation (preventing perplexity values above $\sim 5 \times 10^8$) is a reasonable guard against numerical instability in early training or degenerate cases. The ablation study trains a second full decoder from scratch without retrieval context, which is the correct experimental methodology — reusing trained weights would conflate the effect of retrieval with the effect of additional training steps. The percentage improvement in perplexity is reported and saved to a JSON file.

5. Bonus — Hyperparameter Tuning Log

The hyperparameter tuning section is one of the stronger parts of the submission. Five encoder configurations (E1 through E5) and five decoder configurations (D1 through D5) are documented with their key hyperparameters and approximate validation results. The experiments are designed thoughtfully: E1 tests an under-capacity model to establish a lower baseline, E2 is the chosen configuration, E3 explores a larger model to test whether adding capacity is worth the computational cost, E4 examines the effect of higher dropout, and E5 tests a higher learning rate.

The Key Findings section that follows the tables is particularly valuable. It translates the raw numbers into actionable conclusions — for example, that E2 to E3 gives less than 1% accuracy gain for a 4x increase in parameters at this dataset scale, or that $k=5$ for RAG adds noise rather than improving perplexity. These are genuine analytical observations, not just repetitions of the table values. The RAG ablation study, which trains a complete second decoder without retrieval context, is also correctly implemented and directly quantifies the PPL improvement attributable to the retrieval component.

6. Summary:

```
=====
ASSIGNMENT 3 – FINAL RESULTS SUMMARY
=====

🏠 Part A – Encoder (Multi-Task Transformer)
Sentiment | Accuracy: 0.8654 Macro-F1: 0.4887
Len Bucket | Accuracy: 0.9998 Macro-F1: 0.9997
Params      : 3,105,670

🏠 Part B – Retrieval Module
Index size  : 25,200 vectors x 128d
Metric      : Cosine similarity (pre-normalised dot product)
k chosen    : 3

🏠 Part C – Decoder (RAG Pipeline)
Test PPL (with RAG) : 1.6113
Test PPL (no RAG)   : 1.1160
PPL improvement      : -0.4953 (-44.4 %)
Params               : 3,088,640
```